

Modelo Singleton

O Singleton tem como propósito garantir que uma classe/objeto tenha apenas uma única instância ativa durante a execução do programa, assim, ele fornece um único ponto de acesso global para essa instância.

O Singleton resolve dois problemas :

1. Garantir uma única instância de uma classe, ou seja, ele impede que novos objetos sejam criados se já existir um ativo, reaproveitando a instância existente e evitando desperdício de memória e processamento.
2. Fornecer um ponto de acesso global, semelhante a uma variável global onde é possível acessar o objeto de qualquer parte do código. Mas é mais seguro que uma variável global convencional porque protege a instância de ser acidentalmente sobrescrita por outro trecho do código.

A implementação de Singleton baseia-se em dois passos:

1. Tornar o construtor padrão privado para que outros objetos não possam utilizar o operador new diretamente na classe.
2. Criar um método estático de criação: Este método funciona como o construtor público. Ele chama o construtor privado, armazena o objeto criado em um campo estático e, em chamadas futuras, retorna sempre essa mesma instância que foi salva em cache.

Prós:

Garante o controle estrito de que a classe terá apenas uma instância.

Oferece um ponto de acesso global e seguro.

A instância só é criada e inicializada quando for requisitada pela primeira vez (economizando recursos).

Contras:

Viola o Princípio de Responsabilidade Única (pois resolve dois problemas de uma só vez).

Pode mascarar um design ruim na aplicação, especialmente se os componentes começarem a saber demais uns sobre os outros devido ao acesso global facilitado.

Exige tratamento especial em ambientes multithread para garantir que duas threads diferentes não criem instâncias concorrentes ao mesmo tempo.

Dificulta a realização de testes unitários, já que muitos frameworks de teste dependem de herança para simular objetos (mocking), e o Singleton possui construtor privado e métodos estáticos difíceis de sobrescrever.

Código Implementado(TextureManager)

O código implementa o padrão de projeto Singleton (através da classe TextureManager) em C++. O objetivo do Singleton é garantir que uma classe tenha apenas uma única instância ativa na memória durante toda a execução do programa e fornecer um ponto de acesso global a ela. Neste caso, ela funciona como um gerenciador de texturas de um video game.

Ele está dividido em três arquivos: TextureManager.h, TextureManager.cpp e main.cpp.

Para executar os arquivos existe um Makefile com os comandos make (para compilar e rodar o código) e make clean para deletar o executável após compilação.

1. Cabeçalho (TextureManager.h)

Define a estrutura da classe e aplica as restrições necessárias para que ela não possa ser instanciada livremente:

- Construtor Privado (private: TextureManager());: O código impede que qualquer outra parte do programa crie uma instância usando TextureManager gerenciador; ou new TextureManager().
- Operações de Cópia Deletadas (= delete):

```
TextureManager(const TextureManager&) = delete;  
TextureManager& operator=(const TextureManager&) = delete;
```

Isso impede que a instância única seja copiada ou atribuída a outra variável (por exemplo, TextureManager copia = TextureManager::getInstance();).
- Ponto de Acesso Global (static TextureManager& getInstance());: É um método estático público. Como é estático, ele pode ser chamado diretamente através da classe (TextureManager::getInstance()) sem a necessidade de um objeto previamente criado. Ele retorna uma referência para a única instância existente.

2. Implementação (TextureManager.cpp)

Aqui é onde utilizamos uma abordagem conhecida em C++ como **Meyers' Singleton**:

```
TextureManager& TextureManager::getInstance() {  
    static TextureManager instance;  
    return instance;  
}
```

- `static TextureManager instance;`: O modificador `static` dentro de um método garante que essa variável seja inicializada apenas uma vez (na primeira vez que a função for chamada) e permaneça viva na memória até o fim do programa.
- Segurança de Threads: A partir do C++11, a inicialização de variáveis estáticas locais é garantida pelo padrão da linguagem como sendo thread-safe (segura para multi-threading). Se duas threads tentarem chamar `getInstance()` ao mesmo tempo pela primeira vez, o C++ garante que o construtor será executado apenas uma vez de forma segura.
- Mensagens de Log: O construtor exibe uma mensagem indicando quando o objeto está sendo efetivamente criado na memória, facilitando a visualização de que ele só roda uma vez.

3. Fluxo de Execução (main.cpp)

O arquivo principal serve para demonstrar o funcionamento prático do padrão:

1. Primeiro Acesso:

```
TextureManager& gerador1 = TextureManager::getInstance();
```

Como é a primeira chamada, a variável estática `instance` dentro de `getInstance()` é criada. O construtor privado é executado e a mensagem "[CONSTRUTOR] Instancia criada na memoria!" é impressa. A textura "player.png" é carregada e o endereço de memória de `gerador1` é exibido.

2. Segundo Acesso:

```
TextureManager& gerador2 = TextureManager::getInstance();
```

Nesta chamada, a instância estática já existe. Portanto, o construtor não é chamado novamente. O programa simplesmente retorna uma referência para o mesmo objeto já existente. A textura "enemy.png" é carregada e o endereço de memória de `gerador2` é exibido.

3. Verificação de Igualdade:

```
if (&gerador1 == &gerador2) { ... }
```

O programa compara os endereços de memória de `gerador1` e `gerador2`. Como ambos apontam para o mesmo local de memória, o programa confirma que ambos são exatamente o mesmo objeto.

Link para o código do site refactoring.guru utilizado como exemplo:

<https://refactoring.guru/pt-br/design-patterns/singleton/cpp/example>